

PMSCS 670

Software Testing

Model-Driven Test Design And Coverage Criteria

Lecture 3

Mohammad Ashraful Islam

Coverage Criteria (2.4)

- Even small programs have too many inputs to fully test them all
 - **double** computeAverage (**int** A, **int** B, **int** C)
 - On a 32-bit machine, each variable has over 4 billion possible values
 - Over **80 octillion possible tests!!**
 - Input space might as well be infinite
- This is the source of two key problems in testing:
 - (1) how do we search? and
 - (2) when do we stop?
- Testers **search** a huge input space
 - Trying to find the **fewest inputs** that will find the **most problems**

Coverage Criteria (2.4)

- **Coverage criteria** give structured, practical ways to search the input space
- Satisfying a coverage criterion gives a tester some amount of confidence in two crucial goals:
 - **Search** the input space thoroughly
 - Not much **overlap** in the tests
- Coverage criteria have many advantages for improving the quality and reducing the cost of test data generation.

Test Requirements and Criteria

- **Test Criterion** : A collection of rules and a process that define test requirements
 - Cover every statement
 - Cover every functional requirement
- **Test Requirements** : Specific things that must be satisfied or covered during testing
 - *Each statement might be a test requirement*
 - *Each functional requirement might be a test requirement*

Testing researchers have defined dozens of criteria, but they are all really just a few criteria on four types of structures ...

1. Input domains
2. Graphs

3. Logic expressions
4. Syntax descriptions

Old View : Colored Boxes

- **Black-box testing(BBT)** : Derive tests from external descriptions of the software, including specifications, requirements, and design
- **White-box testing(WBT)** : Derive tests from the source code internals of the software, specifically including branches, individual conditions, and statements
- **Model-based testing(MBT)** : Derive tests from a model of the software (such as a UML diagram)

MDTD makes these distinctions less important.

The more general question is:

from what abstraction level do we derive tests ?

White-box vs Black-box Testing

- White-box testing a.k.a. **structural testing**
- Examines source code with focus on:
 - Control flow
 - Data flow
- Control flow refers to flow of control from one instruction to another. Data flow refers to propagation of values from one variable or constant to another variable
- It is applied to individual units of a program
- Software developers perform structural testing on the individual program units they write
- Black-box testing a.k.a. **functional testing**
- Examines the program that is accessible from outside
- Applies the input to a program and observe the externally visible outcome
- It is applied to both an entire program as well as to individual program units
- It is performed at the external interface level of a system. It is conducted by a separate software quality assurance group

Types of Test Activities

- Testing can be broken up into **four** general types of activities
 1. **Test Design** 
 2. **Test Automation**
 3. **Test Execution**
 4. **Test Evaluation**
- Each type of activity requires different **skills**, background **knowledge**, **education** and **training**
- No reasonable software development organization uses the same people for requirements, design, implementation, integration and configuration control

Why do test organizations still use the same people for all four test activities??

This clearly wastes resources

1. Test Design—(a) Criteria-Based

Design test values to satisfy coverage criteria or other engineering goal

- This is the **most technical** job in software testing
- Requires **knowledge** of :
 - Discrete math
 - Programming
 - Testing
- Requires much of a **traditional CS** degree
- This is **intellectually** stimulating, rewarding, and challenging
- Test design is analogous to **software architecture** on the development side
- Using people who are not qualified to design tests is a sure way to get **ineffective tests**

1. Test Design—(b) Human-Based

Design test values based on domain knowledge of the program and human knowledge of testing

- This is much **harder** than it may seem to developers
- **Criteria-based approaches** can be **blind to special situations**
- Requires **knowledge** of :
 - Domain, testing, and user interfaces
- Requires almost **no traditional CS**
 - A background in the **domain** of the software is essential
 - An **empirical background** is very helpful (biology, psychology, ...)
 - A **logic background** is very helpful (law, philosophy, math, ...)
- This is **intellectually** stimulating, rewarding, and challenging
 - But **not to typical CS majors** – they want to solve problems and build things

2. Test Automation

Embed test values into executable scripts

- This is slightly **less technical**
- Requires knowledge of **programming**
- Requires very **little theory**
- Often requires solutions to difficult problems related to **observability** and **controllability**
- Can be **boring** for test designers
- **Programming is out of reach** for many **domain experts**
- Who is responsible for determining and embedding the expected outputs ?
 - **Test designers may not** always **know the expected outputs**
 - **Test evaluators** need to get involved early to help with this

3. Test Execution

Run tests on the software and record the results

- This is **easy** – and trivial if the tests are well automated
- Requires basic **computer skills**
 - Interns
 - Employees with no technical background
- Asking qualified test **designers** to execute tests is a sure way to convince them to look for a **development job**
- If, for example, GUI tests are not well automated, this requires a lot of **manual labor**
- Test executors have to be very **careful** and **meticulous** with bookkeeping

4. Test Evaluation

Evaluate results of testing, report to developers

- This is much **harder** than it may seem
- Requires **knowledge** of :
 - Domain
 - Testing
 - User interfaces and psychology
- Usually requires almost **no traditional CS**
 - A background in the **domain** of the software is essential
 - An **empirical background** is very helpful (biology, psychology, ...)
 - A **logic background** is very helpful (law, philosophy, math, ...)
- This is **intellectually** stimulating, rewarding, and challenging
 - But **not to typical CS majors** – they want to solve problems and build things

Other Activities

- **Test management** : Sets policy, organizes team, interfaces with development, chooses criteria, decides how much automation is needed, ...
- **Test maintenance** : **Save tests for reuse** as software evolves
 - Requires cooperation of test **designers and automators**
 - Deciding when to trim the test suite is partly policy and partly technical – and in general, **very hard** !
 - Tests should be put in **configuration control**
- **Test documentation** : All parties participate
 - Each test must document “**why**” – criterion and test requirement satisfied or a rationale for human-designed tests
 - Ensure **traceability** throughout the process
 - Keep **documentation** in the automated tests

Organizing the Team

- A mature test organization needs **only one test designer** to work with several test automators, executors and evaluators
- **Improved automation** will reduce the number of test executors
 - Theoretically to zero ... but not in practice
- Putting the **wrong** people on the **wrong** tasks leads to **inefficiency**, **low job satisfaction** and **low job performance**
 - A qualified test designer will be **bored** with other tasks and look for a job in development
 - A qualified test evaluator will **not understand** the benefits of test criteria
- Test evaluators have the **domain knowledge**, so they **must** be free to add tests that “blind” engineering processes will not think of
- The four test activities are **quite different**

**Many test teams use the same people for
ALL FOUR activities !!**

Applying Test Activities

**To use our people effectively
and to test efficiently
we need a process that**

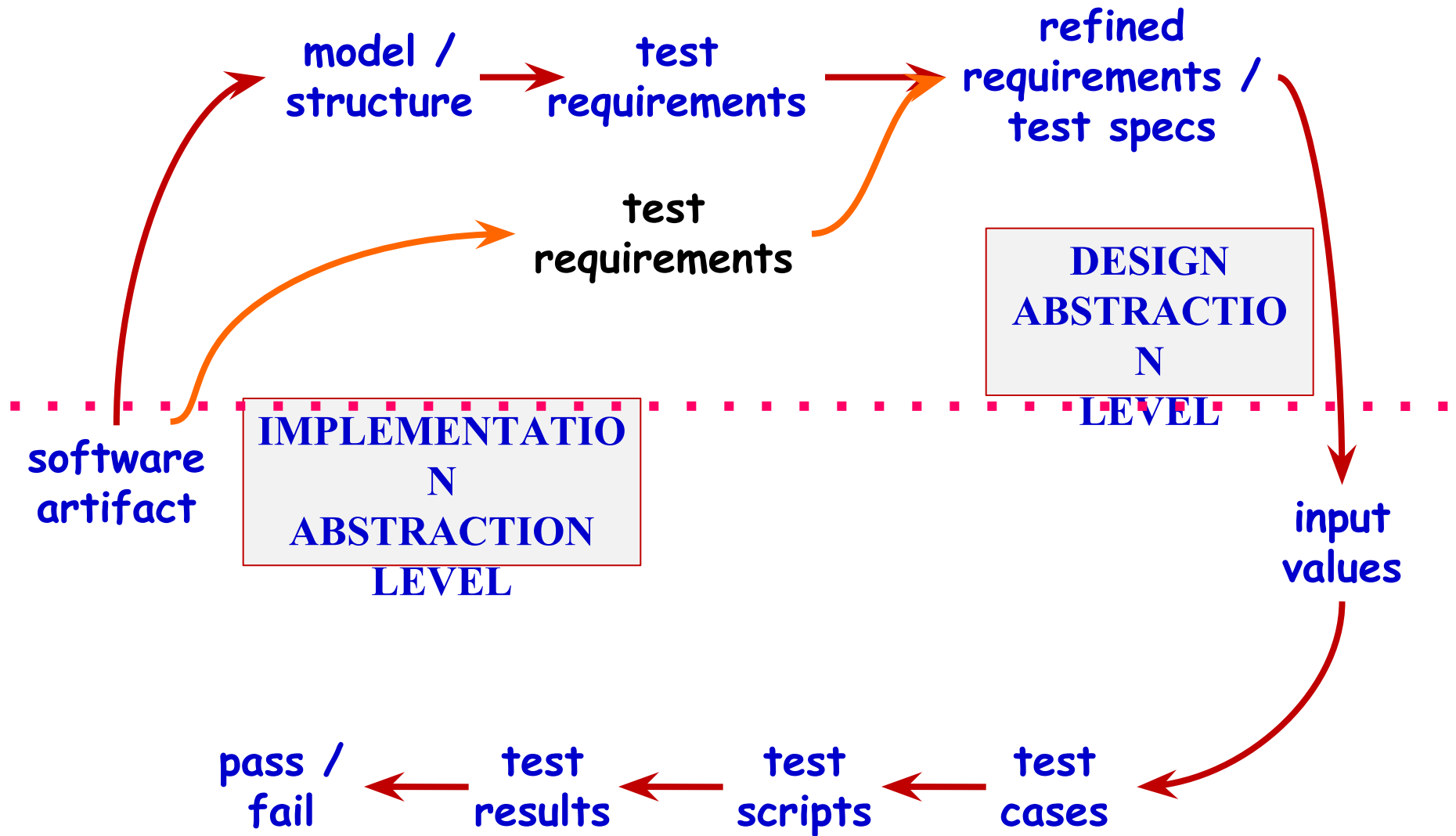
**lets test designers
raise their level of abstraction**

Using MDTD in Practice

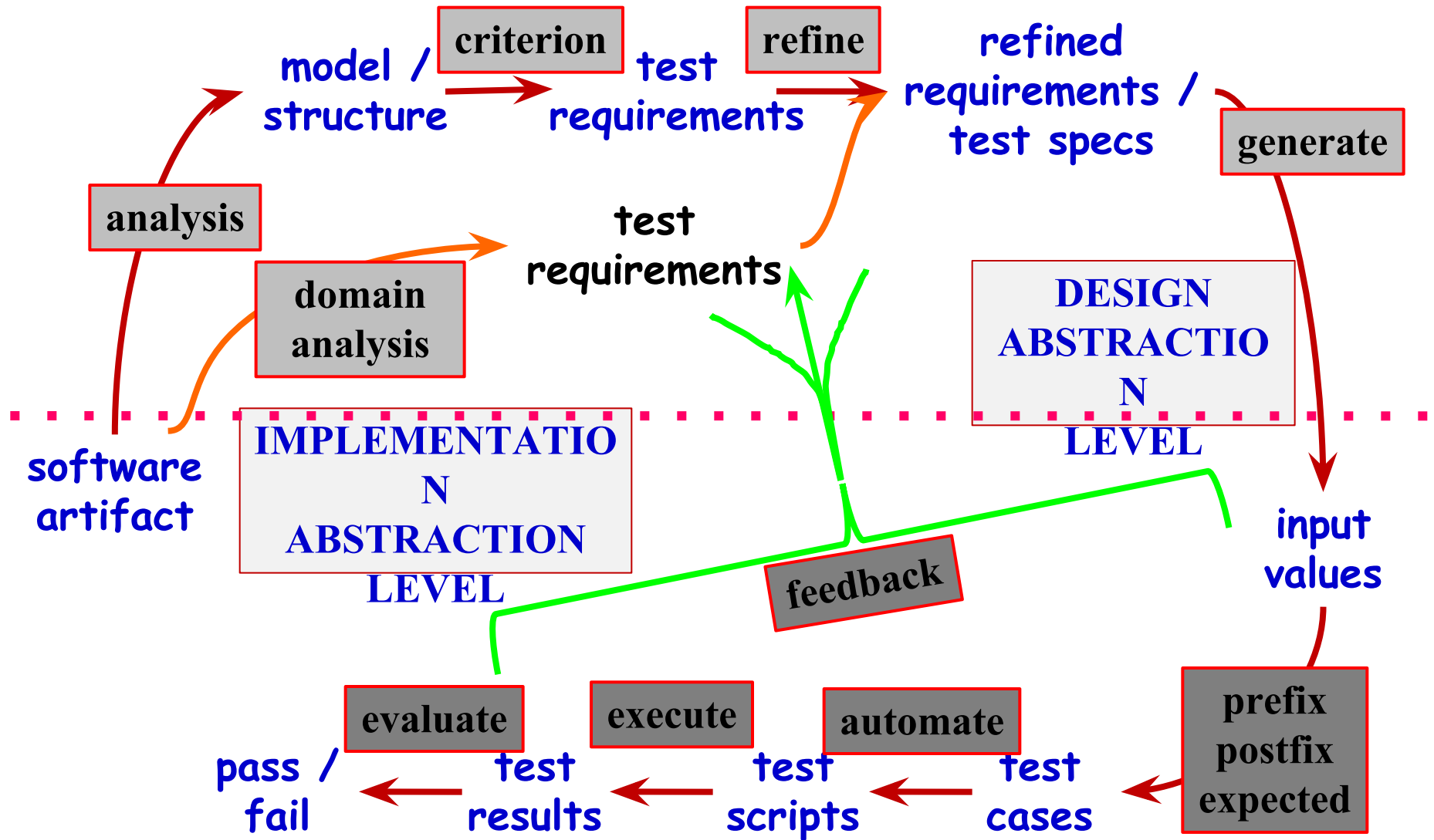
- This approach lets **one test designer** do the math
- Then traditional **testers** and **programmers** can do their parts
 - Find values
 - Automate the tests
 - Run the tests
 - Evaluate the tests
- Just like in **traditional engineering** ... an engineer constructs models with calculus, then gives direction to carpenters, electricians, technicians, ...

Test designers become technical experts

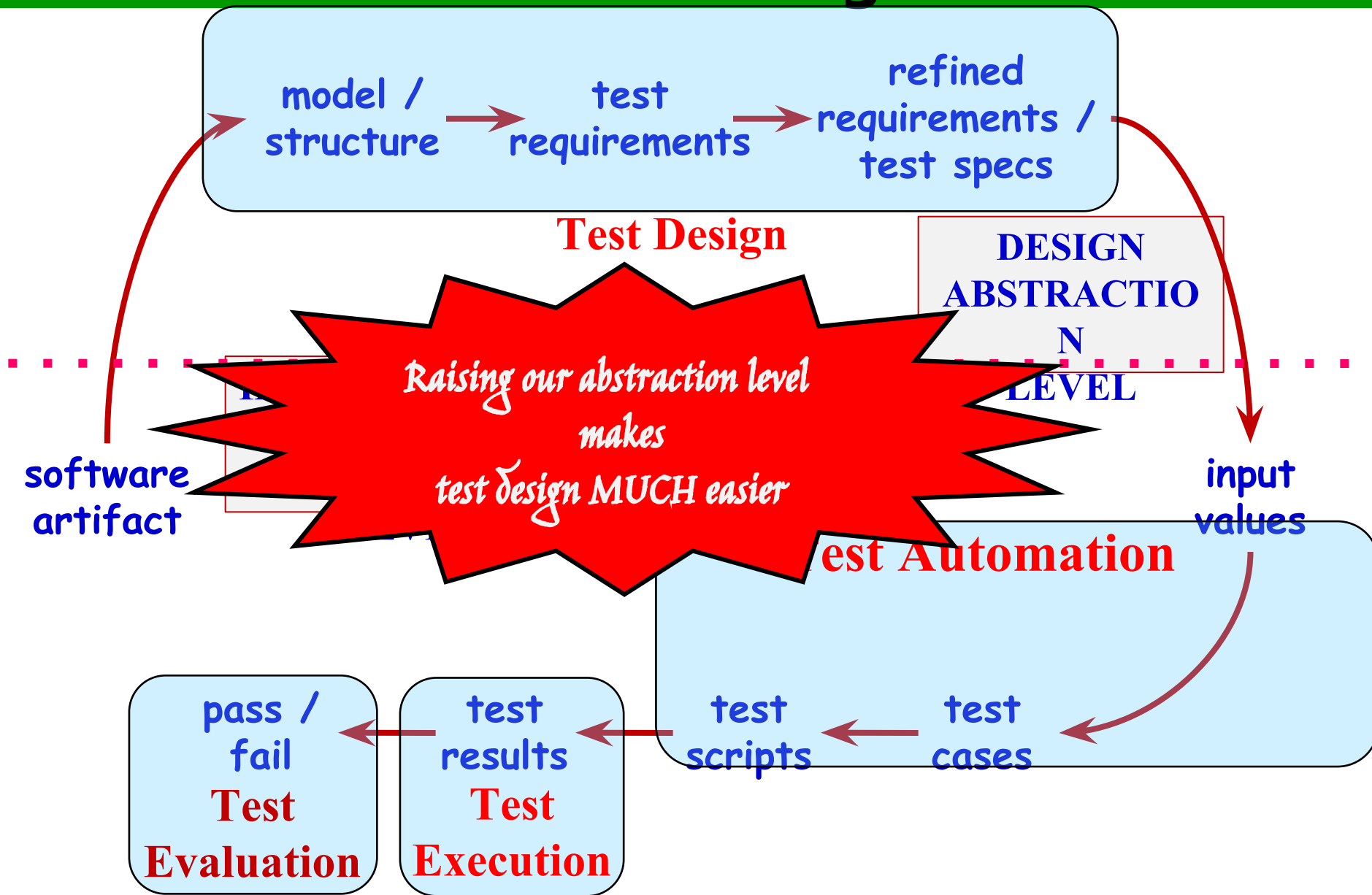
Model-Driven Test Design



Model-Driven Test Design – Steps



Model-Driven Test Design–Activities

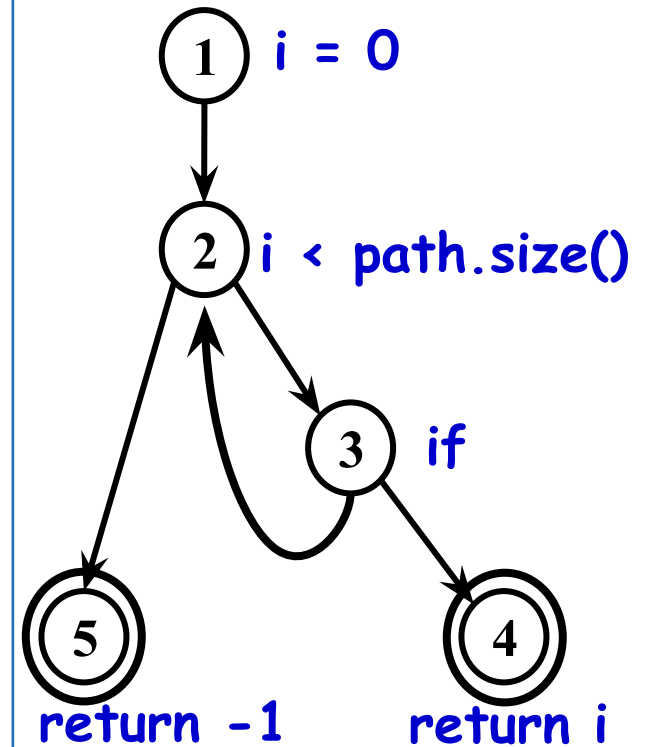


Small Illustrative Example

Software Artifact : Java Method

```
/**
 * Return index of node n at the
 * first position it appears,
 * -1 if it is not present
 */
public int indexOf (Node n)
{
    for (int i=0; i < path.size(); i++)
        if (path.get(i).equals(n))
            return i;
    return -1;
}
```

Control Flow Graph

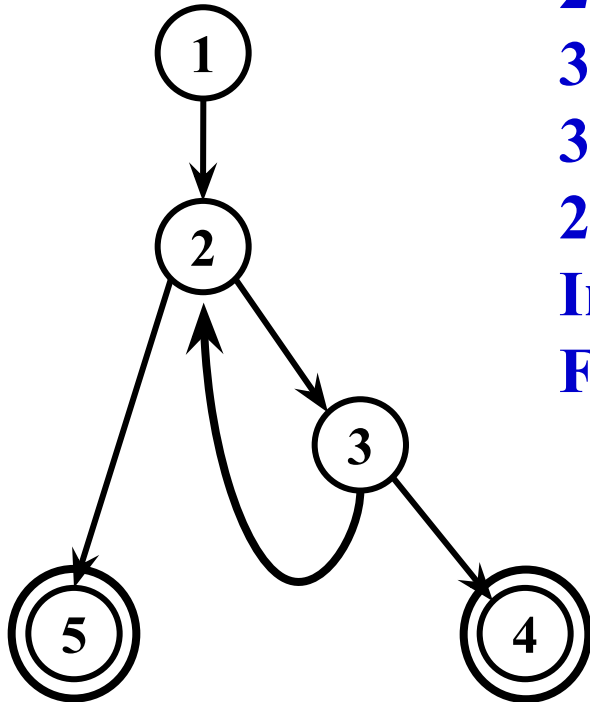


Example (2)

Support tool for graph coverage

<http://www.cs.gmu.edu/~offutt/softwaretest/>

**Graph
Abstract version**



Edges

1 2

2 3

3 2

3 4

2 5

Initial Node: 1

Final Nodes: 4, 5

**6 requirements for
Edge-Pair Coverage**

1. [1, 2, 3]

2. [1, 2, 5]

3. [2, 3, 4]

4. [2, 3, 2]

5. [3, 2, 3]

6. [3, 2, 5]

Test Paths

[1, 2, 5]

[1, 2, 3, 2, 5]

[1, 2, 3, 2, 3, 4]

Find values

...

All Possible Inputs?

- Let's do activity
- Create **all** possible test inputs for the given program

```
/**
 * Determine if the argument is a leap year in the Gregorian calendar
 * Assumes that arguments are in Gregorian calendar range (1582 and onwards)
 *
 * @param year value in range for Gregorian calendar
 * @return true iff year is a leap year
 */
public static boolean isLeap(int year)
{
    if (year % 4 != 0)        return false;
    if (year % 400 == 0)      return true;
    if (year % 100 == 0)      return false;

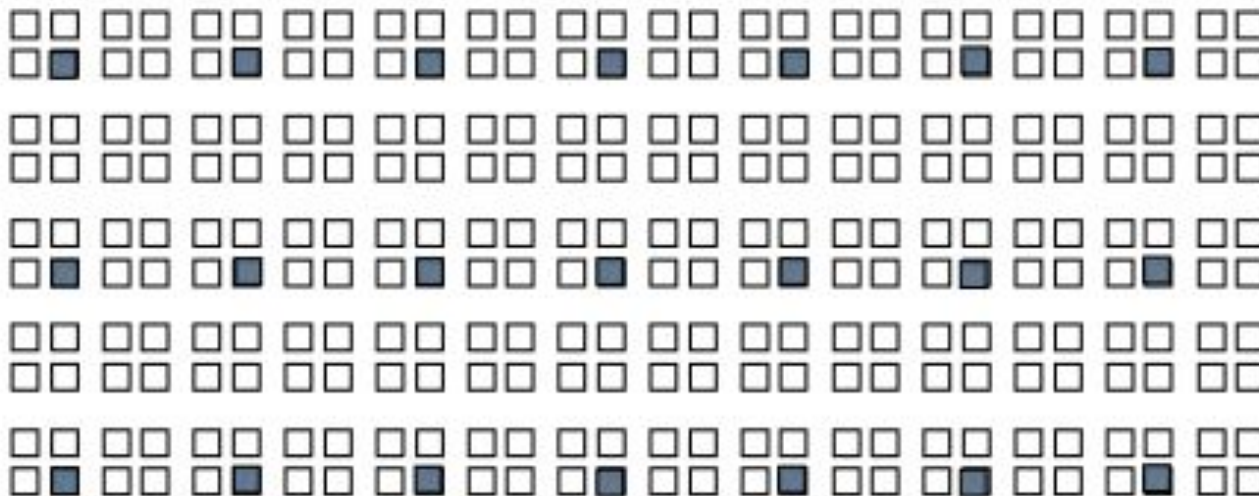
    return true;
}
```

- It is impossible to provide all possible inputs
- Therefore, we need some rules to help us decide which inputs to enter and give us an idea if we test enough

Coverage Criteria

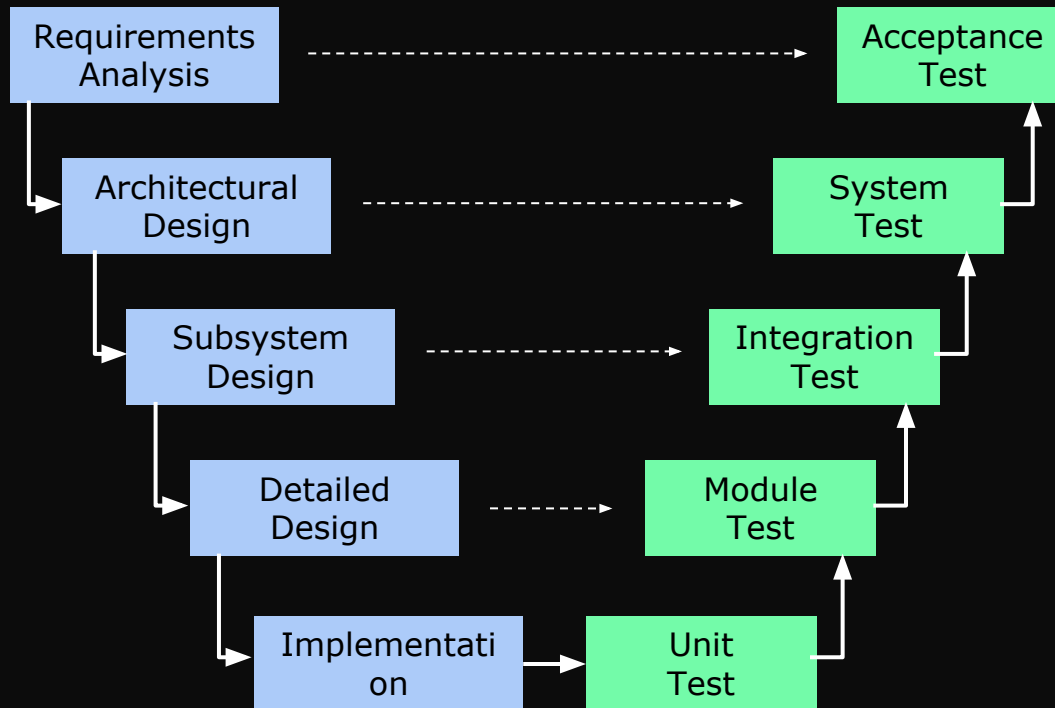
- Describe a finite subset of test cases out of the vast/infinite number of possible tests we should execute
- Divide the input space to maximize the number of faults found per test case
- Provide useful rules for when to stop testing

☐ Possible test case



Changing Notions in Testing

Old view (phase)



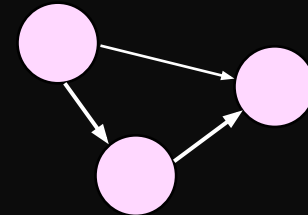
[AO, p 12]

New view (structures and criteria)

Input space (sets)

A: {0, 1, >1}
B: {undergraduate, graduate}
C: {1000, 2000, 3000, 4000}

Graphs



Logical expressions

(not X or not Y) and A and B

Syntax structures (grammar)

```
if (x > y)
  z = x - y;
else
  z = 2 * x;
```

Changing Notions of Testing

- Old view focused on testing at each software development **phase** as being very different from other phases
 - Unit, module, integration, system ...
- New view is in terms of **structures** and **criteria**
 - input space, graphs, logical expressions, syntax
- Test design is largely the same at each phase
 - Creating the model is different
 - Choosing values and automating the tests is different

Benefits of Coverage Criteria

Adequate

- Have I got enough tests?

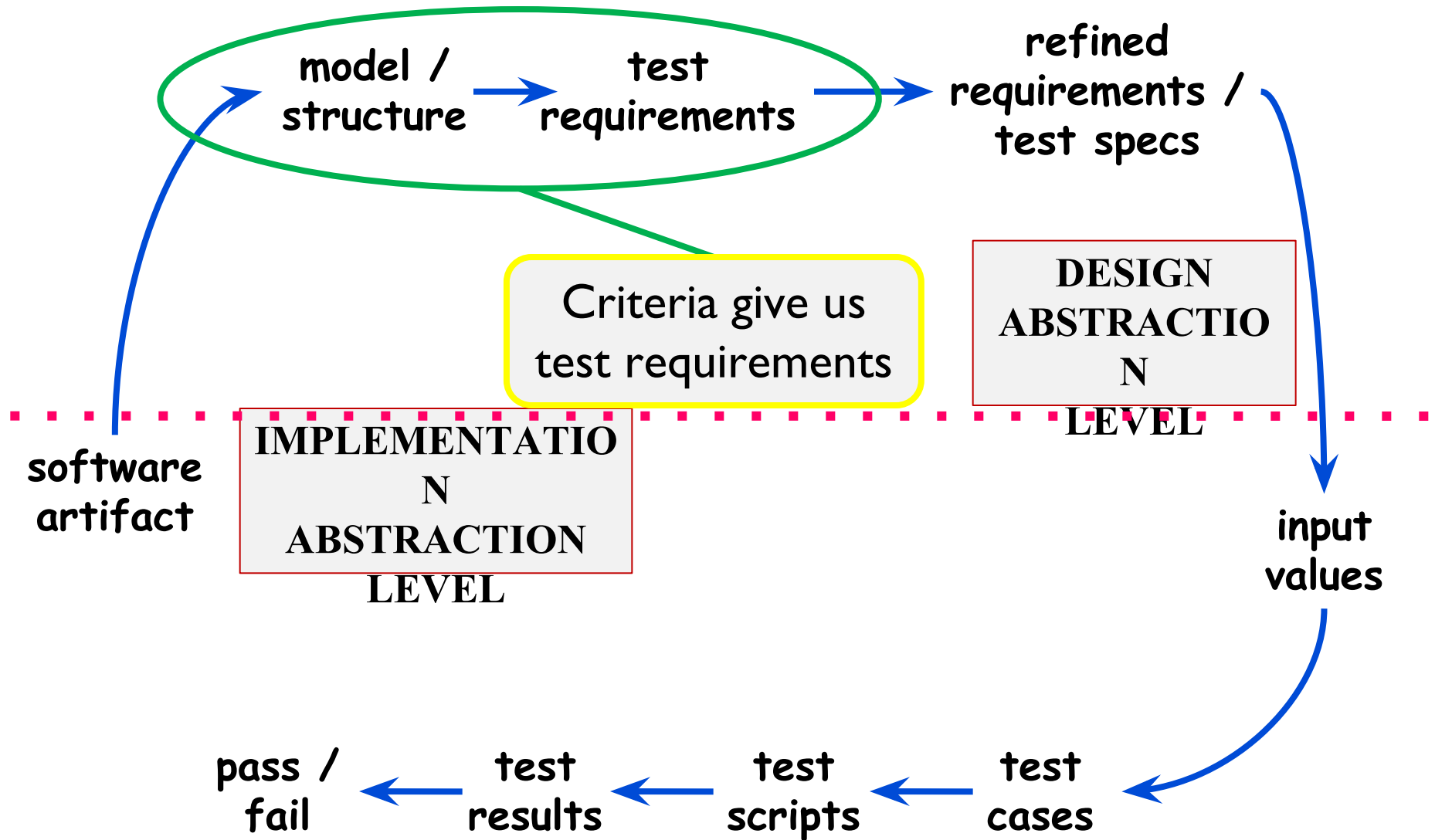
Guidance

- Where should I test more?

Automation

- Generate test that satisfies a test requirement

Model-Driven Test Design



New : Test Coverage Criteria

A tester's job is simple : Define a model of the software, then find ways to cover it

- g **Test Requirements** : A specific element of a software artifact that a test case must satisfy or cover
- g **Coverage Criterion** : A rule or collection of rules that impose test requirements on a test set

Testing researchers have defined dozens of criteria, but they are all really just a few criteria on four types of structures ...

Source of Structures

- These structures can be **extracted** from lots of software artifacts
 - **Graphs** can be extracted from UML use cases, finite state machines, source code, ...
 - **Logical expressions** can be extracted from decisions in program source, guards on transitions, conditionals in use cases, ...
- This is not the same as “***model-based testing***,” which derives tests from a model that describes some aspects of the system under test
 - The model usually describes part of the **behavior**
 - The **source** is explicitly **not** considered a model

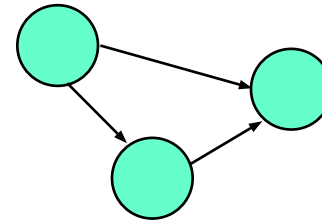
Criteria Based on Structures

Structures : Four ways to model software

1. Input Domain
Characterization
(sets)

A: {0, 1, >1}
B: {600, 700, 800}
C: {swe, cs, isa, ifs}

2. Graphs



3. Logical Expressions

(not X or not Y) and A and B

4. Syntactic Structures
(grammars)

```
if (x > y)
    z = x - y;
else
    z = 2 * x;
```

Example: Source Code

```
public static int numZero(int[] x)
{
    // Effects: if x == null, throw NullPointerException      NPE-B1
    //           else return the number of occurrences of 0 in x
    int count = 0;      B1      // line1
    for (int i=1; i<x.length; i++) {      // line2
        if (x[i] == 0) B2      // line3
            count++;      // line4
    }
    return count;      // line5
}
```

Test requirements for **line coverage**

$TR = \{\text{line1, line2, line3, line4, line5}\}$

Test requirements for **branch coverage**

$TR = \{\text{NPE-B1, B1, !B1, B2, !B2}\}$

Coverage

Given a set of test requirements TR for coverage criterion C , a test set T satisfies C coverage if and only if for every test requirement tr in TR , there is at least one test t in T such that t satisfies tr

- **Infeasible test requirements** : test requirements that cannot be satisfied
 - No test case values exist that meet the test requirements
 - Example: Dead code
 - Detection of infeasible test requirements is formally undecidable for most test criteria
- Thus, 100% coverage is **impossible** in practice

Infeasible Test Requirement

Example:

```
/**
 * @param s1, s2, s3: sides of the putative triangle
 * @return enum describing type of triangle
 */
public static Triangle triangle (int s1, int s2, int s3)
{
```

Imagine if we have the following test requirements

TR = {all sides > 0, all sides = 0, all sides < 0}

Example : Jelly Bean Coverage

Flavors :

1. Lemon
2. Pistachio
3. Cantaloupe
4. Pear
5. Tangerine
6. Apricot



Colors :

1. Yellow (Lemon, Apricot)
2. Green (Pistachio)
3. Orange (Cantaloupe, Tangerine)
4. White (Pear)

• Possible coverage criteria :

1. Taste one jelly bean of each flavor
 - Deciding if yellow jelly bean is Lemon or Apricot is a controllability problem
2. Taste one jelly bean of each color

More Jelly Beans

T1 = { three Lemons, one Pistachio, two Cantaloupes, one Pear, one Tangerine, four Apricots }

- Does test set T1 satisfy the flavor criterion ?

T2 = { One Lemon, two Pistachios, one Pear, three Tangerines }

- Does test set T2 satisfy the flavor criterion ?
- Does test set T2 satisfy the color criterion ?

Coverage Level

The Coverage Level is the ratio of the number of test requirements satisfied by T to the size of TR

- T2 on the previous slide satisfies 4 of 6 flavor test requirements

Two Ways to Use Test Criteria

1. **Directly generate** test values **to satisfy** the criterion
 - Often assumed by the research community
 - Most obvious way to use criteria
 - Very hard without automated tools
2. Generate test values **externally** and **measure** against the criterion
 - Usually favored by industry
 - Sometimes misleading
 - If tests do not reach 100% coverage, what does that mean?

**Test criteria are sometimes called
metrics**

Generators and Recognizers

- **Generator** : A procedure that automatically generates values to satisfy a criterion
- **Recognizer** : A procedure that decides whether a given set of test values satisfies a criterion
- Both problems are provably undecidable for most criteria
- It is possible to recognize whether test cases satisfy a criterion far more often than it is possible to generate tests that satisfy the criterion
- Coverage analysis tools are quite plentiful

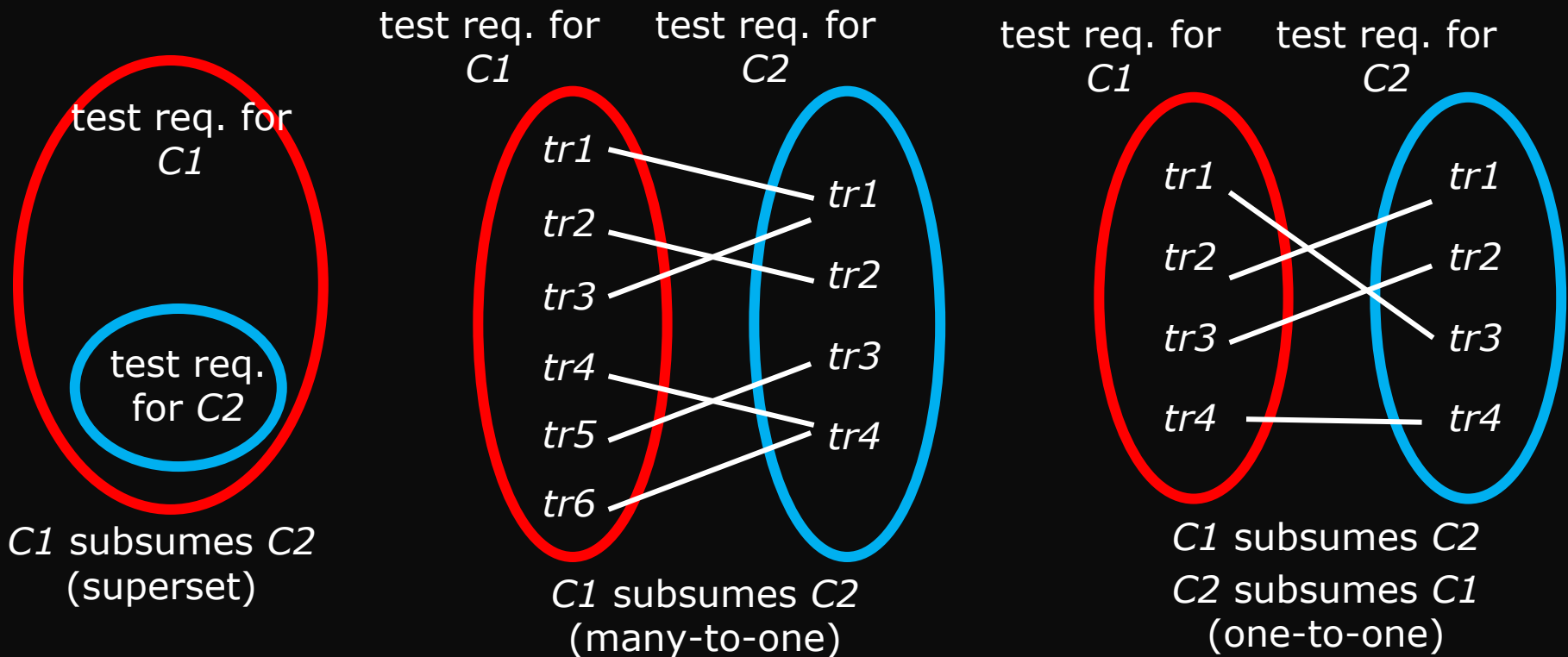
Comparing Criteria

- **Criteria Subsumption** : A test criterion $C1$ subsumes $C2$ if and only if every set of test cases that satisfies criterion $C1$ also satisfies $C2$
- Must be true for every set of test cases
- *Examples* :
 - The **flavor criterion** on jelly beans **subsumes the color criterion** ... if we taste every flavor we taste one of every color
 - If a test set has covered every branch in a program (satisfied the branch criterion), then the test set is guaranteed to also have covered every statement

Comparing Criteria

Criteria Subsumption

- A test criterion $C1$ subsumes $C2$ if and only if every set of test cases that satisfies criterion $C1$ also satisfies $C2$
- Must be true for **every** test set



Characteristics of a Good Coverage Criterion

1. It should be fairly easy to compute test requirements **automatically**
 2. It should be **efficient to generate** test values
 3. The resulting tests should reveal as many **faults** as possible
- Subsumption is only a **rough approximation** of fault revealing capability
 - Researchers still need to gives us more data on how to **compare** coverage criteria

Advantages of Criteria-Based Test Design (5.3)

- Criteria maximize the “bang for the buck”
 - Fewer tests that are more effective at finding faults
- Comprehensive test set with minimal overlap
- Traceability from software artifacts to tests
 - The “**why**” for each test is answered
 - Built-in support for regression testing
- A “**stopping rule**” for testing—advance knowledge of how many tests are needed
- Can be well supported with powerful **tools(automation)**
- Provide greater assurance that the software is of **high quality** and **reliability**
- Yield **fewer tests** that are **more effective** at finding faults
 - Design test inputs that are more likely to find problems

How to Improve Testing ?

- Testers need more and better **software tools**
- Testers need to adopt **practices** and **techniques** that lead to more **efficient** and **effective** testing
 - More education
 - Different management organizational strategies
- Testing / QA teams need more **technical expertise**
 - Developer expertise has been increasing dramatically
- Testing / QA teams need to **specialize** more
 - This same trend happened for development in the 1990s

Four Roadblocks to Adoption

1. Lack of test education

Microsoft and Google say half their engineers are testers, programmers test half the time

Number of UG CS programs in US that require testing ? **0**

Number of MS CS programs in US that require testing **0**

Number of UG testing classes in the US ? **~50**

2. Necessity to change process

Adoption of many test techniques and tools require changes in development process

This is expensive for most software companies

3. Usability of tools

Many testing tools require the user to know the underlying theory to use them

Do we need to know how an internal combustion engine works to drive ?

Do we need to understand parsing and code generation to use a compiler ?

4. Weak and ineffective tools

Most test tools don't do much – but most users do not realize they could be better

Few tools solve the key technical problem – **generating test values automatically**

Needs From Researchers

1. **Isolate** : Invent processes and techniques that isolate the theory from most test practitioners
2. **Disguise** : Discover engineering techniques, standards and frameworks that disguise the theory
3. **Embed** : Theoretical ideas in tools
4. **Experiment** : Demonstrate economic value of criteria-based testing
 - Which criteria should be used and when ?
 - When does the extra effort pay off ?
5. **Integrate** high-end testing with development

Needs From Educators

1. **Disguise** theory from engineers in classes
2. **Omit** theory when it is not needed
3. **Restructure** curricula to teach more than test design and theory
 - Test automation
 - Test evaluation
 - Human-based testing
 - Test-driven development

Changes in Practice

1. **Reorganize** test and QA teams to make effective use of individual abilities
 - One math-head can support many testers
2. **Retrain** test and QA teams
 - Use a process like MDTD
 - Learn more testing concepts
3. **Encourage** researchers to embed and isolate
 - **Invent** processes and techniques
 - **Embed** theoretical ideas in tools
 - Demonstrate economic value of criteria testing
 - Which criteria should be used and when?
 - When does the extra effort pay off?
4. **Get involved** in curricular design efforts through industrial advisory boards

Criteria Summary

- Many companies still use “monkey testing”
 - A human sits at the keyboard, wiggles the mouse and bangs the keyboard
 - No automation
 - Minimal training required
- Some companies automate human-designed tests
- But companies that use both automation and criteria-based testing

Save money

Find more faults

Build better software

Structures for Criteria-Based Testing

